

Deploy no Cloud Run — Guia para o Grupo de MLOps

Objetivo: Implantar os microsserviços `search-svc` e `report-svc` no Google Cloud Run e acompanhar os logs em tempo real durante a apresentação.

Pré-requisitos

Antes de começar, verifique se as ferramentas estão instaladas:

```
gcloud --version # Google Cloud SDK
docker --version # Docker (usado no build via Cloud Build)
make --version   # Make (para os atalhos do Makefile)
```

1. Configuração Inicial do GCP

1.1 Login e seleção do projeto

```
# Autenticar no Google Cloud
gcloud auth login

# Configurar o projeto (substitua pelo seu Project ID)
gcloud config set project SEU_PROJECT_ID

# Confirmar projeto ativo
gcloud config get-value project
```

1.2 Habilitar as APIs necessárias

```
gcloud services enable \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com
```

Por que essas APIs? - `run.googleapis.com` — para criar os serviços no Cloud Run -
`cloudbuild.googleapis.com` — para compilar a imagem Docker via `--source` -
`artifactregistry.googleapis.com` — para armazenar as imagens geradas

2. Estrutura dos Microserviços

```
microservices/
├── search-svc/      ← API de busca semântica
│   ├── main.py      ← FastAPI app com endpoint POST /search
│   ├── Dockerfile    ← Usa uv para instalar dependências
│   └── pyproject.toml
└── report-svc/      ← API de geração de relatórios
    ├── main.py      ← FastAPI app com endpoint POST /generate
    ├── Dockerfile
    └── pyproject.toml
```

Cada microserviço expõe dois endpoints: - `GET /health` — verifica se o serviço está de pé -
`POST /search` (*search-svc*) ou `POST /generate` (*report-svc*) — endpoint principal

3. Deploy

3.1 Deploy de todos os microserviços (recomendado)

```
make deploy
```

Ou diretamente via script:

```
bash scripts/deploy.sh
```

O script realiza, em ordem: 1. Verifica autenticação e projeto configurado 2. Deploy do `search-svc` via `gcloud run deploy --source` 3. Deploy do `report-svc` via `gcloud run deploy --source` 4. Health check de ambos os serviços 5. Exibe as URLs finais no terminal

3.2 Deploy individual (útil para re-deploy durante a apresentação)

```
# Apenas o search-svc
make deploy-search
```

```
# Apenas o report-svc
make deploy-report
```

3.3 Sobrescrever a região (opcional)

Por padrão, os serviços são deployados em `us-central1`. Para usar outra região:

```
make deploy REGION=southamerica-east1
```

4. Verificar URLs e Status

Após o deploy, para ver as URLs dos serviços:

```
make urls
```

Saída esperada:

```
search-svc: https://search-svc-xxxxxxxxx-uc.a.run.app
report-svc: https://report-svc-xxxxxxxxx-uc.a.run.app
```

Para verificar se os serviços estão respondendo corretamente:

```
make health
```

Saída esperada:

```
🔍 Verificando health checks...
✅ search-svc: OK (https://search-svc-xxxxxxxxx-uc.a.run.app)
✅ report-svc: OK (https://report-svc-xxxxxxxxx-uc.a.run.app)
```

5. Visualizar Logs em Tempo Real

Esta é a parte mais importante para a **apresentação ao grupo**. Mostre os logs enquanto o agente faz chamadas aos microsserviços.

5.1 Logs de um serviço específico

```
# Logs do search-svc
make logs-search
```

```
# Logs do report-svc
make logs-report
```

5.2 Logs de ambos em paralelo

```
make logs-all
```

Cada linha será prefixada com o nome do serviço:

```
[search-svc] POST /search → 200 OK (38ms)
[report-svc] POST /generate → 200 OK (12ms)
```

Use `ctrl+c` para interromper.

5.3 Via gcloud diretamente (mais opções)

```
# Últimas 50 linhas de log
gcloud run services logs read search-svc --region us-central1 --limit 50

# Streaming em tempo real
gcloud run services logs tail search-svc --region us-central1

# Filtrar por severidade
gcloud run services logs tail report-svc --region us-central1 --log-filter "severity>=WARNING"
```

6. Atualizar o `.env` Local

Após o deploy, copie as URLs para o seu arquivo `.env` para testar localmente com os serviços no Cloud Run:

```
# Ver as URLs
make urls

# Editar o .env
SEARCH_SVC_URL=https://search-svc-xxxxxxxxx-uc.a.run.app
REPORT_SVC_URL=https://report-svc-xxxxxxxxx-uc.a.run.app
GOOGLE_API_KEY=sua-chave-aqui
```

7. Roteiro para a Apresentação

Sugestão de fluxo para o grupo de MLOps:

Terminal 1 — Executar o orquestrador:

```
make run-api
```

Terminal 2 — Acompanhar logs do Cloud Run:

```
make logs-all
```

Terminal 3 (opcional) — Frontend:

```
make run-frontend
```

O que mostrar ao grupo

1. **Arquitetura:** Mostre o diagrama no `docs/guia-implementacao-demo.md` — Frontend → Orquestrador → Agentes → Microserviços no Cloud Run.
2. **Deploy ao vivo:** Execute `make deploy-search` para fazer um re-deploy do `search-svc` e mostre o processo de build e push no Cloud Build.
3. **Logs em tempo real:** Com `make logs-all` aberto, dispare uma pergunta no frontend ou curl e mostre as requisições chegando nos microserviços.
4. **Health check:** Execute `make health` para mostrar os serviços respondendo corretamente.
5. **URLs dinâmicas:** Execute `make urls` para mostrar as URLs geradas pelo Cloud Run.

8. Comandos de Referência Rápida

Comando	O que faz
<code>make deploy</code>	Deploy de todos os microserviços
<code>make deploy-search</code>	Deploy apenas do search-svc
<code>make deploy-report</code>	Deploy apenas do report-svc
<code>make logs-search</code>	Logs ao vivo do search-svc
<code>make logs-report</code>	Logs ao vivo do report-svc

Comando	O que faz
<code>make logs-all</code>	Logs de ambos em paralelo
<code>make urls</code>	Exibe as URLs no Cloud Run
<code>make health</code>	Verifica os health checks

9. Troubleshooting

✗ **ERROR: (gcloud.run.deploy) PERMISSION_DENIED**

```
# Verificar APIs habilitadas
gcloud services list --enabled | grep -E "run|build|artifact"

# Habilitar o que estiver faltando
gcloud services enable run.googleapis.com cloudbuild.googleapis.com
```

✗ **Não autenticado no gcloud**

```
gcloud auth login
gcloud auth application-default login
```

✗ **health check: FALHA após o deploy**

```
# Ver os últimos logs de erro
gcloud run services logs read search-svc --region us-central1 --limit 30

# Verificar o status do serviço
gcloud run services describe search-svc --region us-central1
```

✗ **Cold start lento na primeira requisição**

Comportamento esperado em serviços com `--min-instances 0`. A primeira requisição pode levar até 3s para inicializar o container. Para evitar isso em produção, use `--min-instances 1`.